

API Key and Secret Leakage Monitor for Public Student Repositories

A Real-Time, Explainable Detection and Alerting Architecture Using GitHub Events, Regular Expressions, and Shannon Entropy

Abstract. Hardcoded API keys, access tokens, database credentials, private keys, and similar secrets can accidentally enter public source-code repositories during ordinary development. Once committed, a credential can remain recoverable through Git history even if it is removed from the latest file. This paper proposes a lightweight, event-driven monitoring platform for public student repositories. The system receives GitHub push events, validates webhook authenticity, retrieves newly changed commit content, and applies a two-stage detector combining configurable regular expressions with Shannon-entropy analysis. Findings are normalized, deduplicated, assigned a confidence level, stored for historical analysis, and delivered to Slack or Discord without exposing the full secret. A web dashboard presents repository, commit, file, type, confidence, and response status. The paper defines the architecture, algorithm, security controls, database design, evaluation plan, test cases, limitations, and a practical hackathon demonstration.

Keywords: secret leakage, API keys, GitHub security, repository monitoring, entropy, regular expressions, webhooks, DevSecOps, credential exposure

1. Introduction

Students increasingly use APIs for cloud hosting, AI services, maps, databases, authentication, payments, and messaging. Under deadline pressure, credentials can be placed directly inside source code and accidentally committed to a public repository. GitHub notes that hardcoded secrets become part of Git history and can remain accessible even after they are removed from the latest version of a file.

The proposed project focuses on a practical question: can a monitoring system detect likely credential exposure immediately after a public student repository receives a new commit and notify the responsible team with enough context to respond, while preventing the monitoring system itself from becoming another source of leakage?

The project is intentionally a transparent educational monitoring layer rather than a replacement for mature commercial secret-scanning products. Its value is the integration of secure event ingestion, explainable detection, alerting, persistence, and a visual security dashboard.

2. Problem Statement

Students publish repositories for assignments, portfolios, internships, open-source work, and hackathons. Secrets may appear in source files, configuration files, notebooks, documentation, or environment templates. A public commit makes such material accessible to anyone who can view the repository.

The proposed monitor must: (1) receive GitHub webhook events or poll for commits; (2) detect likely secrets using regex patterns and Shannon entropy; (3) send alerts containing repository, commit, file, and secret type; and (4) maintain a dashboard of historical findings. The target demonstration is an alert within seconds of a synthetic test commit.

3. Objectives

Monitor selected public GitHub repositories continuously using push events, with polling as a fallback.

Authenticate incoming webhook deliveries before processing them.

Detect known token formats with configurable regular-expression rules.

Use Shannon entropy as a supporting signal for generic high-randomness candidates.

Reduce false positives using context, keywords, allowlists, placeholders, and deduplication.

Never display or send the complete detected secret.

Persist safe finding metadata and generate Slack or Discord alerts.

Measure latency, precision, recall, false-positive rate, and alert delivery success.

4. Background and Related Work

GitHub provides secret-scanning functionality for public repositories and describes detection as pattern matching with additional validity or metadata checks for supported secret types. GitHub also supports custom patterns and push protection. These capabilities establish the importance of credential detection, while this project emphasizes a transparent implementation that students can understand and modify.

Open-source secret-detection tooling such as Gitleaks demonstrates a useful combination of regular expressions, selected capture groups, Shannon entropy, keywords, and allowlists. Recent empirical research also highlights that rule-based detectors can produce false positives and investigates machine-learning approaches for hardcoded-secret detection.

Slack incoming webhooks provide a simple JSON-based notification mechanism. The same notification interface can support Discord or other collaboration platforms.

5. Proposed System Architecture

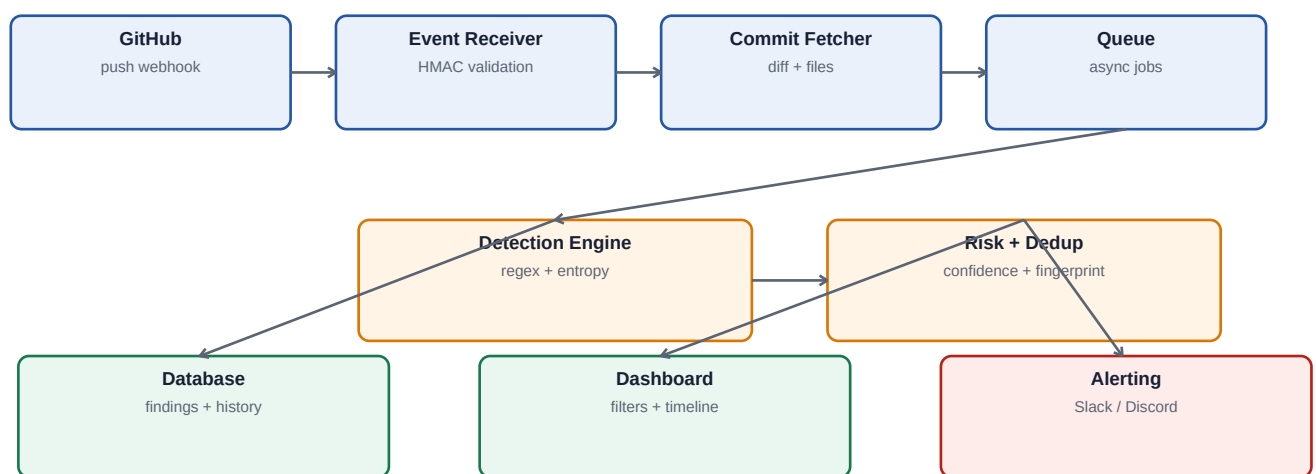


Figure 1. High-level architecture. The webhook receiver validates the GitHub signature before scanning. The worker retrieves changed content, the detector analyzes candidates, and findings are sent to storage, the dashboard, and the alerting layer.

5.1 Event Ingestion

The preferred trigger is the GitHub **push** webhook. GitHub documents that a push event occurs when commits are pushed to a repository branch and includes the SHA of the commit after the push and information about pushed commits. The application should subscribe only to required events and move expensive scanning work into an asynchronous worker.

5.2 Webhook Security

Incoming webhook requests must be authenticated. GitHub signs webhook payloads using a configured secret and exposes the signature through **X-Hub-Signature-256**. The server should calculate HMAC-SHA256 over the raw body and compare it with the received signature using a constant-time comparison. The webhook secret must be stored outside source control.

Recommended sequence: read raw body → verify X-Hub-Signature-256 → verify event type → enqueue event → return quickly → process asynchronously.

6. Secret Detection Methodology

The detector uses two complementary signals. Pattern recognition is effective when a provider token has a recognizable prefix, length, alphabet, or contextual identifier. Shannon entropy measures character uncertainty and can support detection of generated credentials. Neither signal is sufficient alone; context and allowlisting are necessary.

6.1 Regular Expressions

Provider-aware rules should be preferred where token formats are known. Generic rules can detect assignments such as `API_KEY`, `SECRET`, `TOKEN`, or `PASSWORD` followed by a quoted high-randomness value. Production rules should be

configuration-driven so that formats can be updated without changing application code.

Synthetic example only — the following pattern is for testing and is not a real credential:

```
(?i)(api[_-]?key|secret|token|password)\s*[:=]\s*['"]([A-Za-z0-9_\-+/\=]{20,120})['"]
```

6.2 Shannon Entropy

For a candidate string X with character probabilities $p(c)$, Shannon entropy is defined as:

$$H(X) = - \sum p(c) \log_2 p(c)$$

High entropy can indicate a generated secret, but it can also describe a UUID, hash, identifier, compressed data, or other legitimate opaque value. Therefore entropy is best treated as a confidence signal.

6.3 Explainable Combined Score

An explainable MVP score can combine normalized pattern, entropy, and context signals:

$$\text{Score} = 0.55 \times \text{PatternMatch} + 0.30 \times \text{EntropyScore} + 0.15 \times \text{ContextScore}$$

These weights are starting points for experimentation, not universal constants. The team should tune thresholds using a synthetic validation dataset.

6.4 False-Positive Reduction

- Suppress obvious placeholders such as YOUR_API_KEY, example_token, and documented dummy values.
- Use path-based exclusions for generated files, binaries, and known fixtures.
- Apply keyword pre-filters before expensive regex evaluation.
- Require contextual indicators for generic high-entropy strings.
- Maintain explicit allowlists with a reason.
- Deduplicate repeated observations using a safe fingerprint of the candidate.

7. End-to-End Processing Pipeline

1. GitHub push event → 2. webhook verification → 3. commit/file retrieval → 4. candidate extraction → 5. regex matching → 6. entropy/context scoring → 7. deduplication → 8. database persistence → 9. Slack/Discord alert → 10. dashboard update.

7.1 Commit Retrieval

After receiving a verified event, the worker identifies new commit SHAs and retrieves commit metadata and changed-file information through GitHub REST APIs. GitHub's commit endpoints can provide file-level change information, while repository-content endpoints can retrieve file contents when required. The worker should avoid unnecessary downloads and enforce file-size and rate limits.

7.2 Safe Finding Record

Field	Purpose
finding_id	Unique detection identifier
repository	Owner/repository
commit_sha	Commit associated with finding
file_path	File containing candidate
line_number	Approximate source location
secret_type	Provider or generic category
confidence	Low / Medium / High
secret_fingerprint	Non-reversible correlation value
detected_at	Detection timestamp
status	Open / Acknowledged / Revoked / False Positive
remediation	Recommended response

8. Alerting and Dashboard

Alerts should contain repository, commit SHA, file path, secret category, confidence, timestamp, and a commit link. They must never contain the complete credential. Slack incoming webhooks support posting JSON messages to a configured channel, making Slack suitable for the MVP. Discord can be added through the same notification adapter.

Example safe alert:

```
SECRET LEAK DETECTED
Repository: student-demo/security-lab
Commit: 8f2c1a7
File: src/config.js
Type: Generic API Key
Confidence: HIGH
Action: Revoke/rotate the credential and remove it from Git history.
```

Dashboard pages should include: Overview, Detection History, Finding Detail, Repository View, Analytics, and Settings. The finding detail page can display the matched rule, entropy score, context score, fingerprint, and remediation guidance without revealing the raw value.

9. Suggested Technology Stack

Layer	Recommended choice	Purpose
Frontend	Next.js / React + Tailwind	Security dashboard
Backend	FastAPI (Python)	Webhook and REST APIs
Detection	Python regex + entropy module	Explainable scanning
Database	PostgreSQL	Findings and history
Queue	Redis + Celery/RQ (optional)	Async scanning

Layer	Recommended choice	Purpose
GitHub	Webhooks + REST API	Events and commit retrieval
Alerts	Slack / Discord webhooks	Real-time notifications
Deployment	Docker + cloud VM/PaaS	Reproducible deployment

10. Threat Model and Security Controls

Threat	Impact	Control
Forged webhook	Fake or poisoned events	HMAC validation + HTTPS
Compromised GitHub token	Unauthorized repository access	Least privilege + rotation
Dashboard leak	Secondary credential exposure	Never store/display raw secrets
False positives	Alert fatigue	Context + entropy + allowlists
False negatives	Missed exposure	Multiple rules + periodic audits
Alert endpoint leak	Unauthorized notifications	Keep webhook URLs server-side
Database compromise	Finding metadata exposure	Access control + encryption
Log leakage	Secret copied into logs	Redaction + structured logging

11. Evaluation Methodology

The proposed evaluation uses a synthetic benchmark containing positive and negative examples. Positive examples should use deliberately fake, non-functional values that resemble token formats. Negative examples should include environment-variable references, UUIDs, hashes, documentation placeholders, URLs, and generated content.

Metric	Definition	MVP goal
Detection latency	Push event to finding creation	< 5 seconds in controlled demo
Alert latency	Push event to notification	< 10 seconds
Precision	$TP / (TP + FP)$	High while retaining coverage
Recall	$TP / (TP + FN)$	High for supported rules
False-positive rate	FP among negative candidates	Measure and reduce
Alert delivery success	Successful / attempted	$\geq 99\%$ controlled demo

12. Test Cases

ID	Test	Expected result
T01	Synthetic API-key assignment pushed	Finding + alert
T02	Normal environment-variable reference	No alert
T03	High-entropy identifier without secret context	No/low confidence
T04	YOUR_API_KEY placeholder	Suppressed
T05	Tampered webhook signature	Rejected
T06	Same finding pushed twice	Deduplicated
T07	Secret removed in next commit	Historical finding retained
T08	Alert endpoint unavailable	Failure recorded + retry
T09	Large binary file	Skipped/bounded
T10	Multiple secrets in one file	Separate findings

13. Hackathon MVP and Demonstration Plan

The team should prioritize a reliable vertical slice over an oversized feature list.

Phase 1 — Foundation: FastAPI backend, PostgreSQL schema, repository registration, webhook endpoint.

Phase 2 — Detection: 8–12 high-value rules, entropy scoring, placeholder suppression, fingerprint deduplication.

Phase 3 — Alerting: implement Slack first, then add Discord through a common adapter.

Phase 4 — Dashboard: overview cards, findings table, finding detail, repository filters, latency.

Phase 5 — Hardening: signature validation, redaction, retries, rate limits, synthetic test repository.

13.1 Killer Demo Flow

1. Open the dashboard showing zero new findings.
2. Show a monitored public test repository.
3. Commit a deliberately fake API key.
4. Push the commit.
5. Within seconds, show the new HIGH-confidence finding.
6. Show the Slack/Discord alert.
7. Open the finding and explain the regex, entropy, and context signals without exposing the raw secret.
8. Push a benign configuration example and demonstrate that it is not flagged.
9. Send a request with an invalid webhook signature and demonstrate rejection.

14. Differentiating Features

Explainable detection: every finding can show which rule and signals contributed to the decision.

Low-latency monitoring: the demo proves detection shortly after a commit rather than requiring a manual scan.

Secret-safe observability: the dashboard and alerts track exposure without reproducing the credential.

Student-focused design: the system targets common portfolio, assignment, internship, and hackathon repositories.

Provider-independent rules: new token families can be added through configuration.

Secure-by-design monitor: webhook authentication, least privilege, redaction, and alert-secret protection are part of the product.

15. Limitations

- Regex and entropy cannot detect every secret, especially obfuscated, split, encrypted, or unusual credentials.
- Entropy produces false positives because many legitimate strings are high-entropy.
- Provider validity checks are outside the core MVP; the system should say 'likely exposed secret' rather than claiming that a credential is active.
- Historical Git scanning can be expensive and should be scheduled separately.
- An alert is not remediation. Exposed credentials must be revoked or rotated and the incident assessed.
- All demonstrations must use synthetic credentials; the team should never intentionally expose a real key.

16. Future Work

- Machine-learning classification for generic secrets with human feedback.
- Safe provider validity checks where APIs support them.
- Full Git-history scanning and exposure-age estimation.
- Pull-request comments and CI/pre-commit prevention.
- Automatic incident tickets and remediation workflows.
- Repository-level risk scoring.

- Credential rotation integrations.
- Support for GitLab, Bitbucket, and self-hosted Git services.

17. Conclusion

This paper presents a practical event-driven architecture for monitoring public student repositories for likely API keys and other hardcoded secrets. The core pipeline connects GitHub push events to authenticated webhook handling, commit retrieval, regex detection, Shannon-entropy analysis, confidence scoring, deduplication, persistence, dashboard visualization, and real-time alerts. The design is explainable, modular, and suitable for a hackathon MVP.

The strongest demonstration is a complete security story: a synthetic secret is committed, the system detects it within seconds, the team receives an alert, the dashboard explains the finding, and the system itself never reveals the secret. Future versions can add machine learning, validity checks, history scanning, CI prevention, and automated remediation.

18. References

- [1] GitHub Docs. Secret leakage risks. <https://docs.github.com/en/code-security/concepts/secret-security/secret-leakage-risks>
- [2] GitHub Docs. Using webhooks. <https://docs.github.com/en/webhooks/using-webhooks>
- [3] GitHub Docs. REST API endpoints for commits. <https://docs.github.com/en/rest/commits>
- [4] GitHub Docs. Supported secret scanning patterns.
<https://docs.github.com/en/code-security/reference/secret-security/supported-secret-scanning-patterns>
- [5] GitHub Docs. Secret scanning. <https://docs.github.com/en/code-security/concepts/secret-security/secret-scanning>
- [6] Gitleaks. Secret scanning using regex and entropy. <https://github.com/gitleaks/gitleaks>
- [7] Soltaniani, F., & Ghafari, M. Learning to detect hardcoded secrets. Empirical Software Engineering, 2026.
<https://link.springer.com/article/10.1007/s10664-026-10929-w>
- [8] Slack Developer Documentation. Sending messages using incoming webhooks. <https://api.slack.com/messaging/webhooks>
- [9] GitHub Docs. Webhook events and payloads. <https://docs.github.com/en/webhooks/webhook-events-and-payloads>
- [10] GitHub Docs. Validating webhook deliveries. <https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries>
- [11] GitHub Docs. REST API endpoints for repository contents. <https://docs.github.com/en/rest/repos/contents>

Research note: This is a proposed-system / research-style paper. Performance values in the evaluation section are targets, not measured experimental results. Replace them with measured results after testing the implemented system.